自动化与机器学习

TRPO

学习笔记

Markdown 排印版

学术讲义 • AcSg999

目录

TRPO **1**
Background 1
quick facts 1
key equations 1
补充：与 lec 10 off-policy actor-critic 对照 2
实际 TRPO update（以上为 theorical） 2
persudo code 伪代码 4

TRPO

keyward: KL divergence, surrogate advantage

spinningup TRPO

Background

TRPO updates policies by taking the largest step possible to improve performance, while satisfying a special constraint on how close the new and old policies are allowed to be. 快速收敛但保持限制

This is different from normal policy gradient, which keeps new and old policies close in parameter space（控制 θ old-new 不要太大）

recap 之前在西湖大学强化学习笔记 lec 9 学过的 policy gradient&policy update：

$$\theta_{t+1} = \theta_t + \alpha \, \nabla_\theta J(\theta)|_{\theta=\theta_t} = \theta_t + \alpha \mathbb{E}_{S\sim d_{\pi_{\theta_t}}, A\sim\pi_{\theta_t}(\cdot|S)} \left[\nabla_\theta \ln \pi(A \mid S, \theta)|_{\theta=\theta_t} \, q_{\pi_{\theta_t}}(S, A) \right].$$

用采样近似期望，并用回报估计 action value 后，得到 stochastic gradient-ascent 更新：

$$\theta_{t+1} = \theta_t + \alpha \, \nabla_\theta \ln \pi(a_t \mid s_t, \theta)|_{\theta=\theta_t} \, q_t(s_t, a_t), \qquad q_t(s_t, a_t) = G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k.$$

普通 policy gradient 通过学习率调节参数更新幅度，但参数变化小有可能动作分布变化大；TRPO 则用 KL 散度**约束新旧策略分布的差异**。

quick facts

- TRPO is an on-policy algorithm.
  - 补充：用当前策略与环境交互，收集一批数据来更新策略；更新后再用新策略采集下一批，而不是长期从历史经验池中抽数据训练。
- TRPO can be used for environments with either discrete or continuous action spaces.
  - 补充：离散动作是从几个选项中选一个，比如 “向左 / 向右”；连续动作是输出数值，比如关节力矩。TRPO 都能用，只需让策略输出对应的动作分布，例如离散动作的分类分布或连续动作的高斯分布。
- The Spinning Up implementation of TRPO supports parallelization with MPI.
  - 补充：MPI 是多个进程之间传递数据的工具。Spinning Up 可以让多个进程各自运行环境、同时采样，再汇总梯度等信息并同步更新同一个策略，以加快训练.。

key equations

令 π_θ 表示参数为 θ 的策略；旧策略为 π_{θ_k} ，本轮更新时 θ_k 固定。TRPO 的更新为：

$$\theta_{k+1} = \arg\max_{\theta} \underbrace{\mathbb{E}_{s,a\sim\pi_{\theta_k}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_k}(a\mid s)}A^{\pi_{\theta_k}}(s,a)\right]}_{\mathcal{L}(\theta_k,\theta)\text{：surrogate objective（代理目标）}}$$

$$\text{s.t.} \quad \underbrace{\mathbb{E}_{s\sim\pi_{\theta_k}}\left[D_{KL}\left(\pi_\theta(\cdot\mid s)\,\|\,\pi_{\theta_k}(\cdot\mid s)\right)\right]}_{\bar{D}_{KL}(\theta\|\theta_k)\text{：新旧策略的平均 KL 散度}} \le \delta.$$

直观理解：用旧策略采集的数据，寻找让代理目标尽可能大的新策略，同时将动作分布的变化限制在 δ 以内；$\delta > 0$ 是允许的变化上限。

补充：KL 散度衡量两个概率分布的差异，定义为

$$D_{KL}(P\|Q) = \mathbb{E}_{x\sim P}\left[\log\frac{P(x)}{Q(x)}\right] \ge 0.$$

分布相同时为 0，一般不对称，即 $D_{KL}(P\|Q) \ne D_{KL}(Q\|P)$。

surrogate objective: $L(\theta_k,\theta) = \mathbb{E}_{s,a\sim\pi_{\theta_k}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a)\right]$

advantage function: $A^{\pi_{\theta_k}}(s,a) = Q^{\pi_{\theta_k}}(s,a) - V^{\pi_{\theta_k}}(s)$

$$\mathbb{E}_{s\sim d_{\pi_{\theta_k}},a\sim\pi_{\theta_k}(\cdot|s)}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_k}(a\mid s)}A^{\pi_{\theta_k}}(s,a)\right] = \mathbb{E}_{s\sim d_{\pi_{\theta_k}},a\sim\pi_\theta(\cdot|s)}\left[A^{\pi_{\theta_k}}(s,a)\right]$$

即使动作样本来自旧策略，我们仍然可以用概率比来估计新策略在该状态下的动作分布对应的旧优势

补充：与 lec 10 off-policy actor-critic 对照

将 TRPO 的 surrogate advantage 求梯度，与 lec 10 加入 baseline 后的 actor 更新放在一起：

$$g_{\text{AC}}(\theta) = \mathbb{E}_{s\sim\rho,\,a\sim\beta(\cdot|s)}\left[\frac{\pi_\theta(a\mid s)}{\beta(a\mid s)}\nabla_\theta\ln\pi_\theta(a\mid s)A^{\pi_\theta}(s,a)\right],$$

$$\nabla_\theta\mathcal{L}(\theta_k,\theta) = \mathbb{E}_{s,a\sim\pi_{\theta_k}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_k}(a\mid s)}\nabla_\theta\ln\pi_\theta(a\mid s)A^{\pi_{\theta_k}}(s,a)\right].$$

两式都有 “概率比 × log probability 梯度 × 优势” 的结构，概率比用于修正给定状态下的动作分布。区别是：lec 10 用行为策略 β 的数据估计目标策略 π_θ 的更新方向；TRPO 用本轮旧策略 π_{θ_k} 的数据，并固定旧优势 $A^{\pi_{\theta_k}}$，只对概率比求导。

实际 TRPO update（以上为 theorical）

recall

$$\theta_{k+1} = \arg\max_{\theta} \underbrace{\mathbb{E}_{s,a\sim\pi_{\theta_k}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_k}(a\mid s)}A^{\pi_{\theta_k}}(s,a)\right]}_{\mathcal{L}(\theta_k,\theta)\text{：surrogate objective（代理目标）}}$$

$$\text{s.t.} \quad \underbrace{\mathbb{E}_{s\sim\pi_{\theta_k}}\left[D_{KL}\left(\pi_\theta(\cdot\mid s)\,\|\,\pi_{\theta_k}(\cdot\mid s)\right)\right]}_{\bar{D}_{KL}(\theta\|\theta_k)\text{：新旧策略的平均 KL 散度}} \le \delta.$$

在 θ_k 附近对 $\mathcal{L}(\theta_k,\theta)$ 作一阶 Taylor 近似、对 KL 约束作二阶 Taylor 近似，得到 approximate optimization problem：

$$\theta_{k+1} = \arg\max_{\theta} \quad \underbrace{g^T(\theta-\theta_k)}_{\approx \mathcal{L}(\theta_k,\theta)\text{（一阶近似）}}$$

$$\text{s.t.} \quad \underbrace{\frac{1}{2}(\theta-\theta_k)^T H(\theta-\theta_k)}_{\approx \bar{D}_{KL}(\theta\|\theta_k)\text{（二阶近似）}} \le \delta.$$

$g = \nabla_\theta \mathcal{L}(\theta_k,\theta)|_{\theta=\theta_k}$ 即代理目标在旧参数 θ_k 处的梯度，表示目标增长的方向；by coincidence, 恰好等于 policy gradient 中 $\nabla_\theta J(\theta)$,try to proof.

$H = \nabla_\theta^2 \bar{D}_{KL}(\theta\|\theta_k)\big|_{\theta=\theta_k}$. H：平均 KL 散度 $\bar{D}_{KL}(\theta\|\theta_k)$ 在旧参数 θ_k 处的 Hessian（二阶偏导数组成的矩阵），描述参数变化引起的 KL 增长。

$$D(\theta) \approx \underbrace{D(\theta_k)}_{\text{零阶项}} + \underbrace{\nabla_\theta D(\theta_k)^T(\theta-\theta_k)}_{\text{一阶项}} + \underbrace{\frac{1}{2}(\theta-\theta_k)^T H(\theta-\theta_k)}_{\text{二阶项}}$$

两者都对新参数 θ 求导，再代入 $\theta=\theta_k$ ；因此在上面的近似优化中，g 和 H 都是固定量。KL 在新旧策略相同时为 0 且梯度为 0 ，所以它的 Taylor 展开从二阶项开始。

近似解：$\theta_{k+1} = \theta_k + \sqrt{\frac{2\delta}{g^T H^{-1} g}}\, H^{-1} g$ 。用拉格朗日对偶可解出上述近似问题，沿 $H^{-1}g$ 方向前进，并将步长缩放到二阶 KL 约束的边界。

Fisher Information Matrix（Fisher 信息矩阵）：

$$F(\theta) = \mathbb{E}_{s\sim d_{\pi_\theta},\, a\sim\pi_\theta(\cdot|s)} \left[\nabla_\theta \log \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s)^T\right].$$

它描述策略概率分布对参数变化的局部敏感程度：沿某个方向改变同样多的参数，分布变化越大，该方向对应的 Fisher 曲率就越大（在通常的正则条件下，这里 $F(\theta_k) = H$ ）。

Natural Policy Gradient（自然策略梯度）：$\widetilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta)$ 。用 F^{-1} 调整普通梯度，让策略分布敏感的方向少走一些、不敏感的方向可以多走一些，从而在相同的局部 KL 变化预算下尽量提高目标；在这里，自然梯度方向就是 $H^{-1}g$ 。

Natural Policy Gradient 的核心思想：不要只考虑参数变化了多少，还要考虑这种参数变化会让策略概率分布变化多少。

TRPO 更新-回溯线搜索：$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{g^T H^{-1} g}}\, H^{-1} g$ 。取 $\alpha \in (0,1)$ 并选择满足实际 KL 约束且使代理目标增大的最小非负整数 j ，逐步缩短步长。

进一步计算此更新，显式计算 H^{-1} 代价很大–> 我们令 $x = H^{-1}g$, 转而求 $Hx = g$ 这个线性方程，直接得到所需的向量 x 。

因为 H 维度很大，用 conjugate gradient algorithm（共轭梯度法）迭代逼近 x 。每轮只需要计算 Hv ，v 是本轮给定的向量。

PyTorch 中可通过两次自动求导直接计算 Hv ：先求 KL 对参数的梯度；第一次求导时将 `create_graph` 设为 `True`，保留可继续求导的计算图。再与固定的 v 做内积，对这个标量求导：

$$Hv = \nabla_\theta \left[(\nabla_\theta \bar{D}_{KL}(\theta\|\theta_k))^T v\right]\big|_{\theta=\theta_k}.$$

这样只计算 Hessian-vector product（Hessian 与向量的乘积），整个过程都不需要显式求出 H 或 H^{-1} .

persudo code 伪代码

1. Input：初始策略参数 θ_0 ，初始价值函数参数 ϕ_0 ；策略为 π_θ ，价值函数为 V_ϕ 。
2. Hyperparameters：KL 上限 δ ，回溯系数 $\alpha \in (0,1)$ ，最大回溯次数 K 。
3. **for** $k = 0, 1, 2, \ldots$ ，重复以下步骤：
4. 用当前策略 $\pi_k = \pi_{\theta_k}$ 与环境交互，收集轨迹集合 $\mathcal{D}_k = \{\tau_i\}$ 。其中 τ_i 是一条轨迹，$|\mathcal{D}_k|$ 是轨迹条数；下文 $\hat{\cdot}$ 表示用这批样本得到的估计。

   $$\mathcal{D}_k = \{\tau_1, \tau_2, \ldots, \tau_N\},$$
   $$\tau_i = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$$

5. 计算每一步的 rewards-to-go（从该步开始的折扣回报）。

   $$\hat{R}_t = \sum_{l=t}^{T} \gamma^{l-t} r_l.$$

6. 基于当前价值函数 V_{ϕ_k} ，用优势估计方法计算 $\hat{A}_t$ ，如 GAE。
7. 估计 policy gradient：

   $$\hat{g}_k = \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)|_{\theta=\theta_k} \hat{A}_t.$$

8. 用 conjugate gradient 求解 $\hat{H}_k \hat{x}_k = \hat{g}_k$ ，得到 $\hat{x}_k \approx \hat{H}_k^{-1} \hat{g}_k$ 。其中 $\hat{H}_k$ 是样本平均 KL 在 θ_k 处的 Hessian.
9. 用 backtracking line search 更新策略：

   $$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{\hat{x}_k^T \hat{H}_k \hat{x}_k}}\, \hat{x}_k.$$

   取 $j \in \{0, 1, \ldots, K\}$ 中，使样本代理目标增大且样本平均 KL 不超过 δ 的最小值；K 为最大回溯次数，若都不满足，则保留 θ_k 。
10. 用均方误差拟合价值函数，让 $V_\phi(s_t)$ 接近 $\hat{R}_t$ ：

    $$\phi_{k+1} = \arg\min_\phi \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2.$$